Forecasting Bike Rental Demand using Machine Learning

UE24CS352A Machine Learning Mini-Project | Problem 87

NAME: M B DIVYADHARSHINI (PES2UG24CS252) NAIJA ARUN HODIGERE (PES2UG24CS296)

1. Problem Statement

Bike-sharing operators must decide how many bikes to keep at their stations and when to move them. Too few bikes lose customers, while too many increase storage and relocation costs, so an accurate forecast of hourly demand is valuable. In this project we predict the total number of bikes rented in a given hour (*count*) for the Capital Bikeshare system in Washington D.C., using only the date, time, calendar information and weather conditions. This is a supervised regression problem, based on the Stanford CS229 (2014) report *Forecasting Bike Rental Demand* by Du, He and Zhechev. We compare a linear baseline with tree-based ensemble models, study which engineered features matter, and deploy the best model in a small Streamlit application.

2. Dataset Details

We use the Kaggle *Bike Sharing Demand* data. The labelled file **train.csv** contains 10,886 hourly records from 2011-01-01 to 2012-12-19 with 12 columns, and **test.csv** contains 6,493 unlabelled rows. In Kaggle's split, the training data covers days 1-19 of every month and the test data covers the remaining days of each month. The inputs are *datetime, season, holiday, workingday, weather, temp, atemp, humidity* and *windspeed*. The target *count* has a mean of 191.6, a median of 145 and a skewness of 1.24, and equals the sum of the columns *casual* and *registered*; these two are therefore excluded, because they would leak the answer into the model. Exploratory analysis shows that demand peaks around 8 am and 5-6 pm (commuting), is lowest overnight, is highest in season 3 and lowest in season 1, rises with temperature and falls with humidity. Average demand is almost identical on every weekday, so the hour of the day matters far more than the weekday alone. The data contained no missing values; the cleaning code still drops invalid timestamps and duplicate rows and median-fills non-numeric values.

3. Approach

Features. A shared preprocessing module turns each record into 17 numeric features: year, month, day, hour, day of week, a weekend flag, a weekday rush-hour flag (7-9 am and 5-7 pm), a weekend-daytime flag (10 am-6 pm), a temperature bucket and the original weather and calendar columns. The *day* feature is dropped inside the model pipeline, because it is 1-19 in training and 20-31 in testing, so it cannot generalise (16 features reach the model). **Target and metric.** Because the target is right-skewed, models are trained on log(1 + count) and evaluated with the root mean squared logarithmic error (RMSLE); RMSE and R^2 are reported on real counts, and predictions are converted back with exp(x) - 1 and clipped at zero. **Validation.** Demand depends on time, so a random split would leak information from the future. We use a chronological split: the first 80% of rows (8,708) for training and the last 20% (2,178) for validation. **Models.** We compare Ridge regression (linear baseline), Random Forest, Gradient Boosting, XGBoost and LightGBM, then tune the best tree model with randomised search over three time-ordered folds (15 settings). The final model is the better of the tuned and untuned version on the validation set, refitted on all labelled data for the demo.

4. Implementation Overview

The project is written in Python (pandas, scikit-learn, XGBoost, LightGBM, Streamlit) and split into modules so that every stage uses identical preprocessing. *preprocessing/data_loader.py* loads and validates the CSV files, and *preprocessing/features.py* cleans the data, builds the features and blocks leakage columns; *scripts/prepare_data.py* and *scripts/run_eda.py* write the processed files and the EDA report. *modeling/utils.py* provides the chronological split, the metrics and a pipeline that drops *day*; *notebooks/02_modeling.ipynb* (run on Google Colab) compares the models, tunes the best one, draws the plots and saves *models/best_model.joblib*; and *scripts/train_final_model.py* rebuilds the final XGBoost model in about a minute. *modeling/predict.py* loads the saved model and turns one set of inputs into a predicted count, and *app/streamlit_app.py* is the user interface where a date, time and weather conditions are entered. Library versions are pinned (scikit-learn 1.6.1, XGBoost 3.4.1) and all seeds are fixed at 42.

5. Results

All models were scored on the same chronological validation set (Table 1). Fig. 1 compares the models (a), shows the features the final model relies on (b) and plots predicted against actual rentals (c).

Model	RMSLE	RMSE	R^2
XGBoost (final)	**0.3079**	**60.51**	**0.9227**
LightGBM	0.3121	61.62	0.9198
XGBoost (tuned)	0.3276	71.29	0.8927
Random Forest	0.3701	80.46	0.8633
Gradient Boosting	0.3885	95.11	0.8090
Ridge (baseline)	0.8230	177.20	0.3369

Table 1. Validation scores (lower RMSLE and RMSE are better).

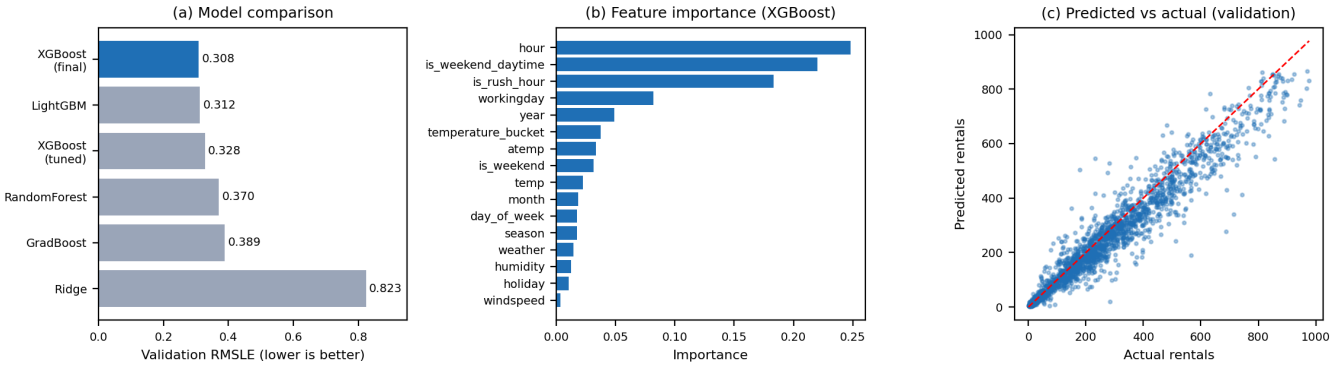


Fig. 1. (a) RMSLE by model, (b) feature importance of the final XGBoost model, (c) predicted vs actual rentals on the validation set (R^2 = 0.92).

Key observations

- **Tree-based models win.** RMSLE is about 0.31-0.39 for the tree models against 0.82 for Ridge, because demand depends non-linearly on hour, temperature and calendar effects. XGBoost is best, closely followed by LightGBM.
- **Engineered features matter.** *hour* (0.25), *is_weekend_daytime* (0.22) and *is_rush_hour* (0.18) are the three most important features, followed by *workingday* and *year*.
- **Tuning did not help.** The tuned XGBoost scored worse on validation (RMSLE 0.3276 vs 0.3079), so the untuned model was kept.
- **Errors are largest at peak demand.** The model under-predicts above roughly 600 rentals, and residuals are widest around the 8 am and 5-6 pm rush hours.

6. Conclusions

Tree-based ensembles predict hourly bike demand far better than a linear model, and XGBoost with calendar-aware features (rush hour, weekend daytime) reached a validation RMSLE of 0.308 and R^2 of 0.92. Using a chronological split, a log target and a leakage guard gave an honest estimate of performance, and the Streamlit demo makes the model easy to use. Limitations are that the model under-predicts very high demand, that test labels are not public so our validation split is the final evaluation (and cannot be compared directly with Kaggle leaderboard scores), and that the data covers only 2011-2012, so for later dates the model treats the year like 2012 and cannot capture long-term growth. Future work includes lag and rolling-demand features, weather interaction features and an ensemble of XGBoost and LightGBM.